62. Chained API Calls: API 2 Needs API 1's Response

Builds on **#33-34 (async/await)**, **#09 (pipes)**, **#24 (exception handling)** and **#61 (waiting vs calculating)**.

> **In one line:** your API calls **API 1** and **waits**, takes a value from its answer, **puts that value into API 2's request**, waits again, then **merges both answers** and returns them to the user.

1. Real-life example: tracking a parcel

You open "Track my order" on a shopping app:
1. The app asks the **Order API**: "Order 101 details?" It answers: "**trackingNo = DEL777**, item = Shoes."
2. Using that trackingNo, the app asks the **Courier API**: "Where is DEL777?" It answers: "In **Pune**, arriving tomorrow."
3. The app **combines both** and shows: "Your shoes are in Pune, arriving tomorrow."

The app **can't ask the courier until the Order API replies**, because it doesn't know the tracking number yet. So the calls must run **one after another**.

2. Flow diagram



What "[await: paused]" means: only **this one function** is paused. Node is **not frozen**. While it waits for the Order or Courier API, it keeps serving other users' requests. Waiting for a network reply is **I/O**, so async/await is enough and worker threads are not needed (#61).

3. The service, with full comments

```
// ─────────────────────────────────────────────────────────────
// tracking.service.ts
// GOAL: GET /track/:orderId
//   API 1 (Order API)   -> gives trackingNo
//   API 2 (Courier API) -> needs that trackingNo, gives location
//   return both combined
// ─────────────────────────────────────────────────────────────
import { Injectable, NotFoundException, BadGatewayException } from '@nestjs/common';

@Injectable() // NestJS creates one shared object of this class (#07 DI)
export class TrackingService {

  // async = this function uses "await" and always returns a Promise
  async trackOrder(orderId: number) {

    // ── API CALL 1: ask the Order API about this order ────────
    // await = pause THIS function until the Order API replies
    // (Node keeps serving other users meanwhile)
    const orderRes = await fetch(`https://orders-api.example.com/orders/${orderId}`);

    // order doesn't exist -> stop here, API 2 is never called
    if (orderRes.status === 404) {
      throw new NotFoundException(`Order ${orderId} not found`);
    }
    // any other failure of the Order API -> 502 (the server we depend on failed)
    if (!orderRes.ok) {
      throw new BadGatewayException('Order service failed');
    }

    // read the body -> e.g. { id: 101, item: "Shoes", trackingNo: "DEL777" }
    // res.json() is async too (the body may still be downloading), so await again
    const order = await orderRes.json();

    // not shipped yet = no tracking number -> no need to call the courier
    if (!order.trackingNo) {
      return { orderId, item: order.item, status: 'NOT_SHIPPED_YET' };
    }

    // ── API CALL 2: DEPENDS on response 1 ─────────────────────
    // we put order.trackingNo (from API 1) into the URL of API 2.
    // This is WHY the calls can't run in parallel: before API 1 answered,
    // we didn't know "DEL777".
    const courierRes = await fetch(
      `https://courier-api.example.com/track/${order.trackingNo}`,
    );
```

```
    // courier down -> still return what we know instead of failing completely
    if (!courierRes.ok) {
      return {
        orderId,
        item: order.item,
        trackingNo: order.trackingNo,
        location: null,
        note: 'Courier service unavailable, try again later',
      };
    }

    // read body 2 -> e.g. { location: "Pune", eta: "2026-11-03" }
    const tracking = await courierRes.json();

    // ── FINAL: combine response 1 + response 2 and return ────
    // NestJS turns this into JSON and sends it with status 200
    return {
      orderId,                      // from the URL
      item: order.item,             // from API 1 (Order API)
      trackingNo: order.trackingNo,  // from API 1, used to call API 2
      location: tracking.location,   // from API 2 (Courier API)
      eta: tracking.eta,            // from API 2
    };
  }
}
```

4. The controller (the URL)

```
// tracking.controller.ts
import { Controller, Get, Param, ParseIntPipe } from '@nestjs/common';
import { TrackingService } from './tracking.service';

@Controller('track')                      // all routes here start with /track
export class TrackingController {
  constructor(private readonly tracking: TrackingService) {}   // NestJS injects the service

  @Get(':orderId')                        // handles GET /track/101
  track(
    @Param('orderId', ParseIntPipe) orderId: number,  // "101" -> 101; "abc" -> 400 Bad Request
  ) {
    return this.tracking.trackOrder(orderId);   // NestJS waits for the Promise, then sends
JSON
  }
}

// tracking.module.ts
@Module({ controllers: [TrackingController], providers: [TrackingService] })
export class TrackingModule {}            // add TrackingModule to AppModule imports
```

5. What the user gets

```
GET http://localhost:3000/track/101

200 OK
{
  "orderId": 101,
  "item": "Shoes",
  "trackingNo": "DEL777",
  "location": "Pune",
  "eta": "2026-11-03"
}
```

6. Timeline of one request

```
  0 ms    request comes in -> controller -> trackOrder(101)
  0 ms    await API 1 (Order)     [paused, Node serves other users]
150 ms    API 1 replies           -> trackingNo = "DEL777"
150 ms    await API 2 (Courier)   [uses DEL777, paused again]
400 ms    API 2 replies           -> location = "Pune"
401 ms    return combined JSON

Total about 400 ms = API 1 time + API 2 time
(one after another, because API 2 depends on API 1)
```

7. What happens when something fails

Situation	What the code does	User gets
Order 101 doesn't exist	Stops after API 1. API 2 is never called.	**404** "Order 101 not found"
Order API crashes (500)	Stops after API 1	**502** "Order service failed"
Order not shipped yet	No trackingNo, so it skips API 2	**200** `status: NOT_SHIPPED_YET`
Courier API down	Returns the order data without a location (fallback)	**200** with `location: null` + a note
Network totally broken	`fetch` throws an error	**500** (NestJS default, #24)
`/track/abc`	ParseIntPipe rejects it before the service runs	**400** Bad Request

8. One after another vs in parallel

Situation	How	Time	Example
API 2 needs API 1's data	`await` API 1, then `await` API 2	A + B	Tracking needs trackingNo from the order (this PDF)
They don't need each other	`Promise.all([a, b])`	max(A, B)	Fees + attendance of the same student

```
// independent calls: run them at the same time
const [fees, attendance] = await Promise.all([
  fetch(`https://fees-api/fees/${id}`).then(r => r.json()),
  fetch(`https://attendance-api/attendance/${id}`).then(r => r.json()),
]);
```

```
  One after another:  [fees 1s][attendance 1s]       = 2 s
  Parallel:           [fees 1s]
                      [attendance 1s]                = 1 s
```

Common mistake: forgetting `await`. `const order = fetch(url)` gives a **Promise**, not the data, so `order.trackingNo` is `undefined` and API 2 is called with `/track/undefined`.

9. Production tips

Tip	Why
Set a **timeout** on every call (e.g. `AbortSignal.timeout(5000)` or `HttpModule.register({ timeout: 5000 })`)	A slow API shouldn't hang your API forever
Return a **fallback** when a non-critical API fails	The user still gets a partial answer
Put API URLs in **.env** (#28, #29 ConfigService)	Different URLs for local, test and prod
Cache responses that rarely change in Redis (#49, #54)	Fewer calls, faster answers
Retry once on network errors (not on 404)	Short network blips get fixed by themselves
Use `@nestjs/axios` `HttpService` in bigger apps	Shared config, interceptors, easy to mock in tests

```
// fetch with a 5-second timeout
const orderRes = await fetch(url, { signal: AbortSignal.timeout(5000) });
```

10. Summary

- **Chained calls** = API 2 needs a value from API 1's response, so they run **one after another**.
- Steps: **await API 1**, check it worked, read `.json()`, **use its value** in API 2's URL, **await API 2**, read `.json()`, **merge and return**.
- `await` pauses **only this function**. Node keeps serving other users.
- Handle failures: 404 if not found, 502 if the other server failed, a **fallback** when a non-critical API is down.
- Independent calls go in `Promise.all`, which is faster. Dependent calls must stay one after another.
- Always **timeout**, keep URLs in **.env**, and **cache** when you can.